UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Doctoral Program in Deep Tech, Artificial Intelligence & Emerging Technologies

Research Methods and Scientific Integrity in AI and Advanced Technologies

REPRODUCIBLE MACHINE LEARNING PIPELINE: SEED ROBUSTNESS, MLFLOW TRACKING AND DATA VERSIONING
DELIVERABLE No. 5

Course Instructor	**Dr. Loveleen Gaur**
Student	**Eng. Jesús Vicente Inca Cárdenas**

2026 - II

Table of Contents

1. Purpose and Deliverable Scope....3
2. Rubric Alignment and Reproducibility Stack....3
3. Dataset and Experimental Design....4
4. Leakage-Free Machine Learning Pipeline....5
5. Repeatability and Multi-Seed Results....6
6. MLflow Experiment Tracking....9
7. Git, DVC, Environment and Stranger Test....9
8. Discussion....11
9. Conclusions....12
10. Limitations and Next Steps....12
References....14
Appendix A. Git Commit Plan....15
Appendix B. Exact Reproduction Commands....15

REPRODUCIBLE MACHINE LEARNING PIPELINE: SEED ROBUSTNESS, MLFLOW TRACKING AND DATA VERSIONING
DELIVERABLE No. 5

Research topic: A Verifiable, Human-in-the-Loop Generative AI Model for Contractual Clause Recommendation in Regulated Fiduciary Institutions. Proxy task for this deliverable: clause-type classification on a public contract corpus.

1. Purpose and Deliverable Scope

This deliverable implements the reproducibility workflow requested in Session 5 of the doctoral course. The rubric requires a Git history with at least six meaningful commits, a dataset versioned with DVC and pushed to a remote, two identical executions with the same seed, mean ± standard deviation over at least five seeds logged in MLflow, direct dependencies pinned with a Dockerfile that matches Python, and a README that allows a stranger to reproduce the reported number. The course frames reproducibility as a stack spanning code, data, experiments, environment, and reporting rather than as a single checkbox (Gaur, 2026).

Instead of the Session 5 workshop dataset, I used a public legal dataset that is directly connected to my doctoral thesis on verifiable clause recommendation in regulated fiduciary institutions. The task is to predict the type of a contract clause (for example, Governing Laws, Notices or Terminations) from its text, using a fixed subset of LEDGAR, the contract-provision corpus included in the LexGLUE benchmark (Tuggener et al., 2020; Chalkidis et al., 2022). Recognising which type of clause is needed is the first step of the artefact designed in Deliverable No. 3, before candidate clauses are retrieved and recommended. LEDGAR contains provisions from contracts filed publicly with the U.S. SEC; no institutional or client data were used, in line with the confidentiality commitments of the research protocol.

The implementation goes beyond the minimum rubric in five ways: seven seeds are evaluated instead of five; a majority-class baseline is evaluated under the same protocol as a minimal analogue of baseline B0 in my protocol; the data and results are produced by a two-stage DVC pipeline (dvc.yaml, dvc.lock, params.yaml) rather than a single tracked file; repeatability is checked both within one process and across fresh processes; and an automated stranger test (src/verify_reproduction.py) compares any new execution with the committed results.

2. Rubric Alignment and Reproducibility Stack

Table 1
Deliverable No. 5 Rubric Alignment

Layer / criterion	Evidence generated	Status
Git history	Eight methodological commits (D5-01 to D5-08) plus one repair commit; the Git commit hash is stored in MLflow and in results/manifest.json	Exceeds 6+ requirement

Layer / criterion	Evidence generated	Status
Data	LEDGAR subset rebuilt from a pinned Hugging Face revision; MD5 pinned in dvc.lock; cache pushed to DVC remote storage	Implemented; cache and remote in sync
Repeatability	Seed 42 run twice in the same process and twice in fresh processes; summary and all five folds identical	Programmatic assertion, passed
Experiment tracking	15 MLflow runs: one parent seed-study run and 14 child runs (2 models × 7 seeds)	Exceeds 5+ requirement
Reporting	Mean ± SD across seven seed-level means; six metrics; baseline under the same protocol	Implemented
Environment	requirements.txt with 14 exact versions; Dockerfile on python:3.13.13-slim, the Python version that produced the results	Implemented
Stranger test	Clean rebuild with dvc repro --force; verify_reproduction.py (data MD5, seed-42 reference, 14 runs); independent re-execution on a different operating system	Passed (Section 7.4)

Note. Individual elaboration.

The architecture follows the weakest-layer principle emphasised in class: perfect code versioning cannot compensate for an unversioned dataset, and exact package versions cannot rescue a leaky preprocessing pipeline. Each layer is therefore treated as independently auditable.

3. Dataset and Experimental Design

Table 2
Dataset and Evaluation Protocol

Item	Specification
Dataset	LEDGAR (LexGLUE benchmark), training split
Source	Hugging Face coastalcph/lex_glue, configuration ledgar, revision c23fdff1a6bf74e0e1a71cb86f1e781d37da888c (pinned in params.yaml)
Subset	The 20 most frequent clause types, 150 clauses each, sampling seed 2026
Observations	3,000 contract provisions (median length 81 words; range 7 to 981)
Classes	Governing Laws, Notices, Counterparts, Entire Agreements, Severability, Survival, Amendments, Assignments, Expenses, Terms, Terminations, Insurances, Taxes, Litigations, Confidentiality, Further Assurances, General, Compliance With Laws, Indemnifications, Waivers
Dataset fingerprint (MD5)	bc995ef9bacb814a07d06dbecad47ad0
Model	TF-IDF (word 1–2 grams, min_df = 2, max 50,000 features, sublinear tf) + logistic regression (lbfgs, C = 4, max_iter = 3,000)
Baseline	DummyClassifier (most frequent class), same folds and seeds
Validation	5-fold stratified cross-validation, shuffled
Seeds	7, 12, 21, 42, 101, 2026, 3407
Metrics	Accuracy, balanced accuracy, macro-F1, weighted F1, macro precision, macro recall

Note. Individual elaboration. Macro-F1 is the primary metric because it weights every clause type equally.

The seed controls the shuffled stratified fold assignment and the model random state. Because the lbfgs solver is deterministic, the variation between seeds measures the effect of data partitioning. The data subset is drawn once with its own seed (2026), so it is identical for every experimental seed. For each seed the reported value is the mean of its five validation folds, and the within-seed standard deviation quantifies fold-to-fold variation. The headline result is the mean ± sample standard deviation of the seven seed-level means, which addresses the course warning that a single run gives no estimate of variance (Gaur, 2026; Pineau et al., 2021).

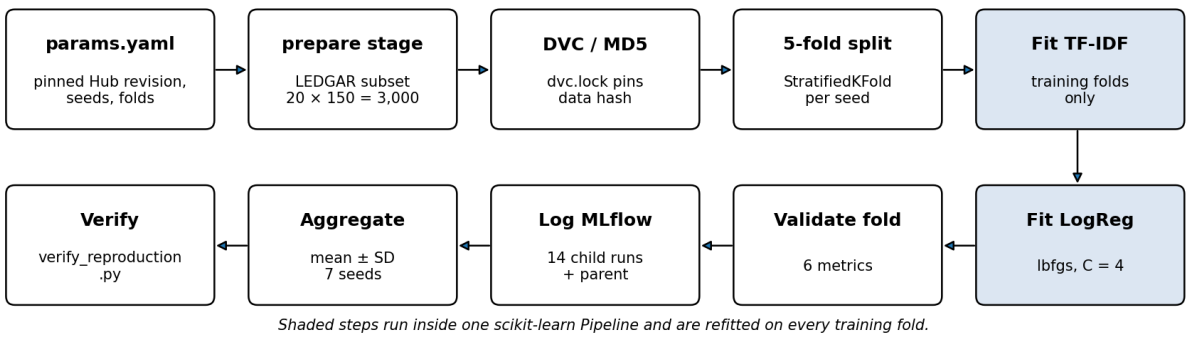
Because every class has exactly 150 clauses and folds are stratified, each validation fold holds 30 clauses per class. Two pairs of metrics therefore coincide by construction: accuracy equals balanced accuracy and macro recall, and macro-F1 equals weighted F1. They are reported for completeness, but they do not add independent evidence.

4. Leakage-Free Machine Learning Pipeline

The central methodological control is a scikit-learn Pipeline containing TfidfVectorizer followed by LogisticRegression (Pedregosa et al., 2011). Because cross-validation calls fit separately inside each training fold, the vocabulary, document frequencies and IDF weights are learned only from training clauses; the validation clauses are transformed with weights they did not help to estimate. Fitting the vectoriser on the full corpus before splitting would let validation text shape the feature space, the kind of preprocessing leakage that Kapoor and Narayanan (2023) show can produce over-optimistic scientific claims.

Hyperparameters are fixed in src/train.py and were not tuned inside the cross-validation, so no model selection leaks into the reported estimates. The majority-class baseline runs through the same Pipeline and folds, which makes the comparison between the two models a like-for-like one.

Figure 1
Leakage-Free Reproducibility Workflow



Note. Author's own elaboration based on the Session 5 reproducibility stack.

5. Repeatability and Multi-Seed Results

5.1 Same-Seed Repeatability Test

Seed 42 was evaluated twice in the same Python process and twice more in fresh processes launched by run_seeds.py. All four result dictionaries were identical, and the five fold accuracies were equal to the last digit: 0.92000000, 0.93166667, 0.93500000, 0.92000000 and 0.91166667. The script raises an error if the two executions differ or if seed 42 no longer matches the committed reference in reference/expected_seed42.json. This satisfies repeatability for the reference execution; it does not by itself prove validity or replicability.

Table 3

Same-Seed Repeatability Evidence

Execution	Seed	Accuracy mean	Macro-F1 mean	Context
Run A	42	0.923667	0.924210	Same process
Run B	42	0.923667	0.924210	Same process
Runs C and D	42	0.923667	0.924210	Two fresh processes
Comparison	—	Identical	Identical	Assertion passed

Note. Individual elaboration, from results/repeatability_seed42.json.

5.2 Seven-Seed Results

Table 4

Performance by Random Seed (TF-IDF + Logistic Regression)

Seed	Accuracy	Fold SD	Macro-F1	Fold SD	Macro precision
7	0.92433	0.00693	0.92479	0.00674	0.92826
12	0.92267	0.01593	0.92285	0.01600	0.92650
21	0.92533	0.01089	0.92566	0.01088	0.92811
42	0.92367	0.00953	0.92421	0.00891	0.92806
101	0.92267	0.00805	0.92329	0.00782	0.92647
2026	0.92233	0.00494	0.92270	0.00447	0.92580
3407	0.92367	0.01556	0.92388	0.01571	0.92625

Note. Individual elaboration. Each value is the mean of five validation folds; fold SD is the sample standard deviation across those folds.

Table 5

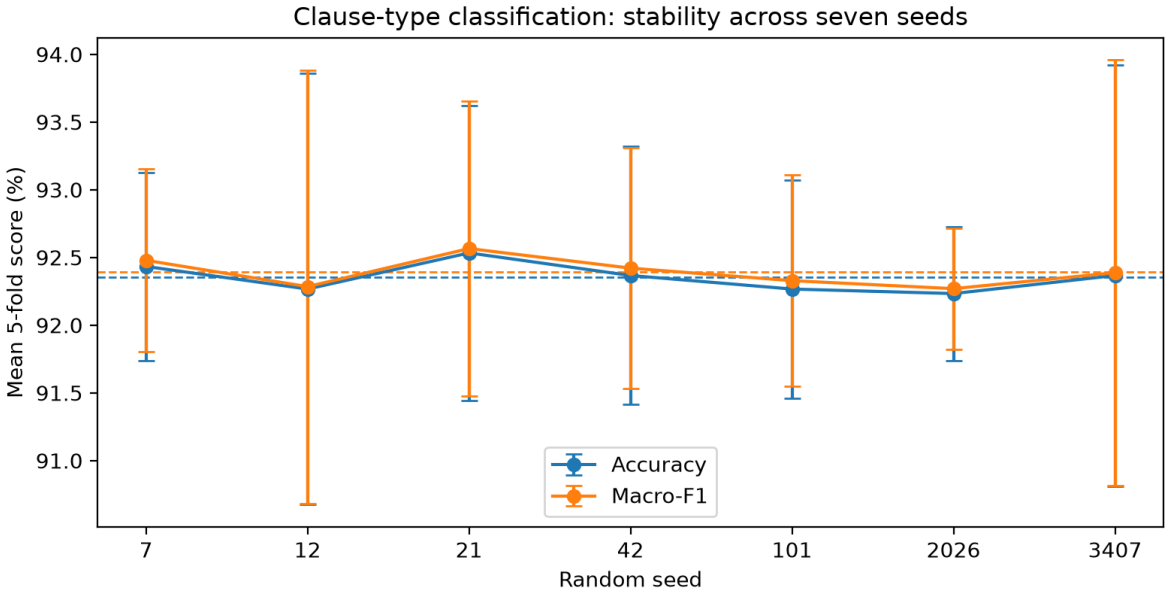
Overall Stability Across Seven Seed-Level Means

Metric	Model	Mean	SD	Min–Max	Range
Accuracy	TF-IDF + LogReg	0.92352	0.00107	0.92233–0.92533	0.30 pp
Macro-F1	TF-IDF + LogReg	0.92391	0.00107	0.92270–0.92566	0.30 pp
Macro precision	TF-IDF + LogReg	0.92706	0.00104	0.92580–0.92826	0.25 pp
Accuracy	Majority baseline	0.05000	0.00000	0.05000–0.05000	0.00 pp
Macro-F1	Majority baseline	0.00476	0.00000	0.00476–0.00476	0.00 pp

Note. Individual elaboration, from results/overall_stability.csv. Balanced accuracy and macro recall equal accuracy, and weighted F1 equals macro-F1, because the classes are balanced (Section 3).

Figure 2

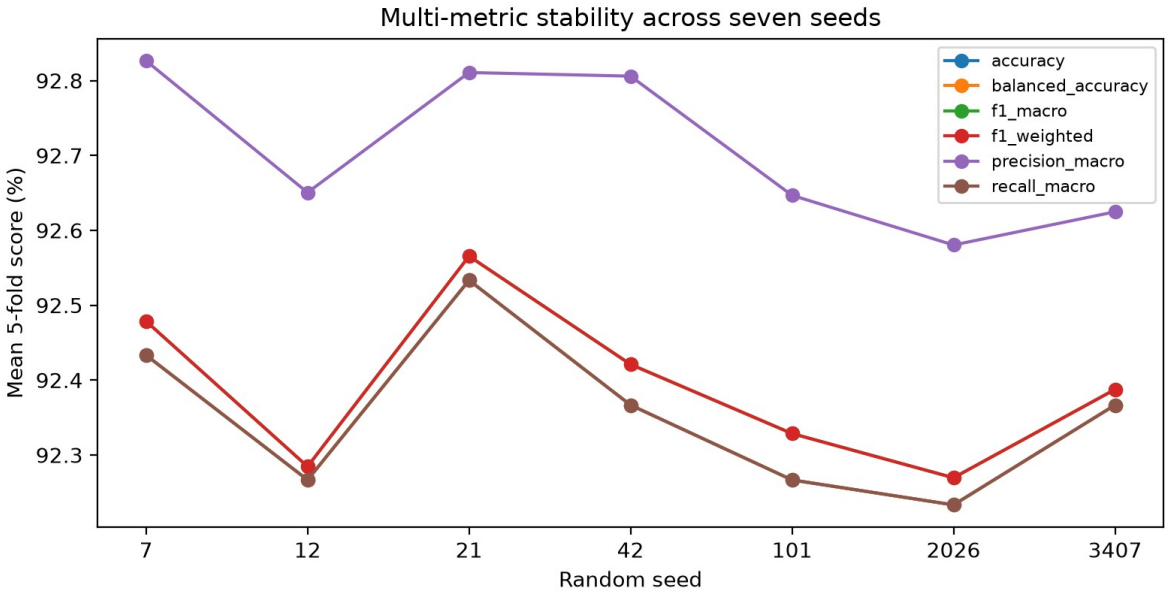
Accuracy and Macro-F1 Across Seven Random Seeds



Note. Error bars show the within-seed standard deviation across five stratified folds. Dashed lines are the means of the seven seed-level values.

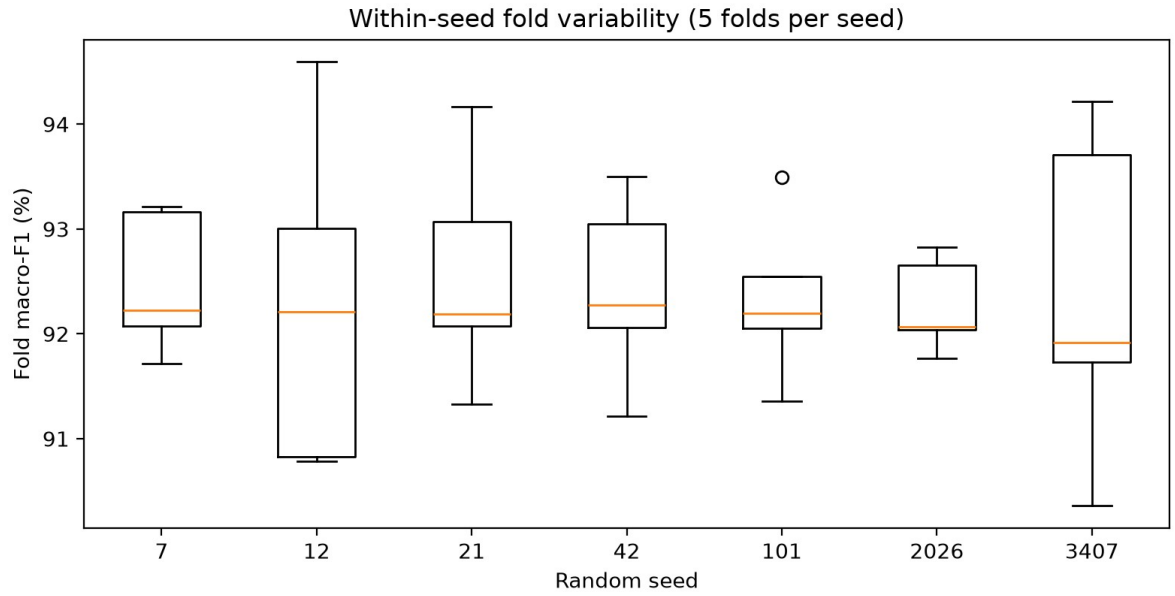
Figure 3

Multi-Metric Stability Across Seven Random Seeds



Note. Each point is the mean of five validation folds for the indicated seed.

Figure 4
Within-Seed Fold Macro-F1 Distribution



Note. Each box summarises the five fold-level macro-F1 values generated by one seed.

5.3 Where the Errors Are: Clause Types

To see what the aggregate hides, the seed-42 cross-validated predictions were broken down by clause type. This analysis was run during the independent re-execution described in Section 7.4, with the same pipeline and folds.

Table 6
Clause Types With the Lowest and Highest F1 (Seed 42)

Clause type	Precision	Recall	F1	Most frequent confusion
General	0.631	0.740	0.681	Terminations, Terms
Terminations	0.828	0.800	0.814	General, Terms
Terms	0.876	0.800	0.836	General (19 of 150)
Waivers	0.928	0.853	0.889	General, Amendments
Amendments	0.893	0.887	0.890	General, Waivers
Governing Laws	0.980	0.987	0.983	—
Severability	0.993	0.980	0.987	—
Counterparts	0.987	0.987	0.987	—

Note. Individual elaboration from cross_val_predict with StratifiedKFold(5, shuffle = True, random_state = 42).

6. MLflow Experiment Tracking

MLflow is configured as the experiment ledger with a SQLite backend (mlflow.db), consistent with the Session 5 material and current MLflow guidance for local database-backed tracking (MLflow Project, 2026). A parent run named seed-study records the seed list and the versions of nine libraries; under it, 14 nested child runs record each model × seed combination. Fold-level CSV files are logged as artifacts, and every run carries the Git commit and a flag indicating whether the working tree had uncommitted changes.

Table 7
MLflow Metadata Logged

Type	Logged content
Parameters (child runs)	seed; model; features; classifier; CV specification; scikit-learn version; data MD5; number of rows; number of classes
Parameters (parent run)	seed list; versions of scikit-learn, NumPy, SciPy, pandas, Matplotlib, MLflow, datasets, DVC and PyYAML
Metrics	Mean and SD of accuracy, balanced accuracy, macro-F1, weighted F1, macro precision and macro recall (12 metrics per run)
Tags	git_commit; git_dirty; Python version; MLflow source tags
Artifacts	fold_metrics_<model>_seed_<seed>.csv; aggregate tables on the parent run
Backend and experiment	sqlite:///mlflow.db; experiment UNMSM-D5-clause-reproducibility; 15 runs, all FINISHED

Note. Individual elaboration, verified by querying mlflow.db.

This design separates repeatability from robustness. A fixed seed permits an exact rerun, while the seven seeds reveal how much the result moves when a legitimate source of randomness, the fold assignment, changes. Reporting that distribution is more defensible than selecting the best seed: seed 21 gives the highest macro-F1 (0.92566), but the reported figure is the seven-seed mean.

7. Git, DVC, Environment and Stranger Test

7.1 Git History

The deliverable folder has eight methodological commits, D5-01 to D5-08, each corresponding to one step of the workflow (Appendix A). They are preceded by a repair commit, D5-00: the folder had first been registered in the outer repository as an embedded repository (a Git link with mode 160000), so its files were not actually versioned. The link was removed and the files were committed stage by stage, which also rebuilt a readable history. The full repository is public at https://github.com/jesusincacardenas-crypto/UNMSM-Research-Methods-2026-II.

The final results were regenerated from commit e7973cb (D5-06) and committed as D5-07. The manifest still records git_dirty = true. The most likely cause is that run_seeds.py inspects the working tree while DVC is rewriting its own outputs during dvc repro, and that Git reported

line-ending normalisation on several files; the source code, parameters and data used are nonetheless those certified by the MD5 hashes in dvc.lock, and the numbers were identical to the earlier run (Section 7.4).

7.2 DVC Data Versioning

Data versioning goes beyond tracking a single file. A two-stage DVC pipeline (dvc.yaml) defines a prepare stage, which rebuilds data/ledgar_subset.csv from the pinned Hugging Face revision, and a seeds stage, which runs the seven-seed study. dvc.lock records the MD5 of every dependency and output: the data file (bc995ef9bacb814a07d06dbecad47ad0), the source scripts, the parameters, and the results and figures directories. dvc status therefore reveals any change in data, code or parameters since the last run, and dvc repro rebuilds only what changed. The CSV is excluded from Git; the results and figures are committed so readers can see the numbers without running anything.

A DVC remote named storage was configured as the default remote, and dvc push sent the data cache to it; dvc status -c reports that the cache and the remote are in sync. The remote is a local directory on the author's machine, which satisfies versioning and recovery but cannot be read by a third party on another computer. That limitation is mitigated by the design of the prepare stage: because the Hugging Face revision is pinned, any reader can rebuild the identical file, and DVC checks it against the locked MD5.

7.3 Environment Freezing

requirements.txt pins the exact versions of 14 direct dependencies (for example scikit-learn 1.9.1, NumPy 2.5.4, pandas 3.1.0, MLflow 3.17.2, DVC 3.67.1). The Dockerfile uses python:3.13.13-slim, the same interpreter that produced the results, sets PYTHONHASHSEED = 0 and limits BLAS and OpenMP to one thread, so that parallel floating-point reductions cannot change the last digits. Its default command runs the full stranger test.

7.4 Stranger Test

The stranger test is automated in src/verify_reproduction.py, which checks three conditions: the data MD5 matches dvc.lock; seed 42 run in a fresh process matches reference/expected_seed42.json to five decimals; and, with --full, all 14 model × seed runs match results/seed_summary.csv within 1e-12.

Three executions confirm that the reported number can be regenerated. First, after all files were committed, dvc repro --force rebuilt the pipeline from scratch: the prepare stage downloaded LEDGAR again from the pinned revision and produced a file with the same MD5 (bc995ef9bacb814a07d06dbecad47ad0); the seeds stage passed the repeatability check, reproduced all seven seed-level means to five decimals (macro-F1 0.92391 ± 0.00107), and reported that seed 42 matched the committed reference.

Second, the committed data and code were re-executed in an independent environment during the preparation of this report: Linux instead of Windows 11, Python 3.12.3 instead of 3.13.13, and scikit-learn 1.8.0 instead of 1.9.1. The data file matched the locked MD5; seed 42 reproduced accuracy 0.92367 and macro-F1 0.92421; and all 14 runs matched the committed seed summary with a maximum absolute difference of 1.1×10^{-16}, which is floating-point rounding. This is a stronger result than the course requires, because it shows the number survives a change of operating system, interpreter and library version, not only a fresh clone. Third, the public repository was cloned into a new directory (C:/temp/stranger_test), a fresh virtual environment was built from requirements.txt, and dvc pull retrieved the data file from the DVC remote (1 file fetched and added). verify_reproduction.py --full then passed all three checks and printed REPRODUCED (Table 8).

Table 8

Stranger Test on a Fresh Clone

Check	Result	Evidence
Data MD5 matches dvc.lock	PASS	bc995ef9bacb814a07d06dbecad47ad0 in both
Seed 42 matches reference/expected_seed42.json	PASS	Accuracy 0.92367; macro-F1 0.92421
All 14 model × seed runs match results/seed_summary.csv	PASS	Maximum absolute difference 0.00 (exact)
Overall verdict	REPRODUCED	Macro-F1 across 7 seeds: 0.9239 ± 0.0011

Note. Individual elaboration, from the console output of verify_reproduction.py --full in the fresh clone.

Taken together, the three executions show the reported number surviving a rebuild of the data from the source, a fresh clone with the data restored from the DVC remote, and a change of operating system, interpreter and library version.

8. Discussion

The principal result is not the macro-F1 of 0.924 itself; it is the quantified stability of that result. Across seven seeds, macro-F1 was 92.39% ± 0.11% and varied by only 0.30 percentage points from minimum to maximum. The within-seed fold standard deviation (0.45 to 1.60 points) is larger than the between-seed standard deviation, which means that a single train–validation split would be a noisier estimate than the seed-level mean. The majority-class baseline (macro-F1 0.005) confirms that the score reflects learned textual signal rather than the class distribution.

The aggregate also hides heterogeneity that matters for the thesis. Clause types with distinctive wording, such as Governing Laws, Severability and Counterparts, reach F1 above 0.98, while General reaches 0.68 and Terminations 0.81, and most errors are confusions among General, Terms and Terminations. These are clause types whose boundaries are semantic rather than

lexical. For a clause-recommendation artefact this means that a correct type label is not enough: two clauses can look alike and differ in legal effect, which supports the protocol decision to evaluate legal correctness separately from attribution.

The deliverable is also a small-scale rehearsal of the evaluation promised in Deliverable No. 3. The protocol commits to k repeated runs per system with fixed seeds, a dated and versioned snapshot of the knowledge base, versioned prompts, seeds and artefacts, and comparison against baselines. Here those commitments are exercised as seven seeds with mean ± SD, a pinned Hub revision with an MD5 locked by DVC, MLflow runs tied to Git commits, and a baseline evaluated under the same protocol. The scaffold can be reused when B0 to B3 are compared on the expert gold standard.

9. Conclusions

- The same-seed test passed: seed 42 reproduced identical summary and fold-level outputs in the same process and in fresh processes.
- The leakage-free Pipeline ensures that TF-IDF weights are learned only from training folds, preventing preprocessing leakage.
- Seven seeds show that the clause-type classifier is stable: macro-F1 92.39% ± 0.11%, against 0.48% for the majority-class baseline.
- Clause types with semantic rather than lexical boundaries (General, Terms, Terminations) concentrate the errors, which supports evaluating legal correctness separately in the thesis.
- MLflow records an auditable history; Git versions code in nine commits; DVC pins data, code and parameters by MD5 and pushes the data to a remote; requirements and Docker freeze the environment; verify_reproduction.py operationalises the stranger test, which printed REPRODUCED on a fresh clone and also passed across operating systems and library versions.
- The pipeline is reusable as the reproducibility scaffold for the evaluation of the thesis artefact.

10. Limitations and Next Steps

This is a reproducibility exercise on a public English-language corpus of U.S. contracts, not an evaluation of the thesis artefact, which will work with Spanish-language fiduciary contracts under Peruvian regulation. Clause-type classification is also simpler than clause recommendation: it does not generate text, cite sources or face hallucination. Cross-validation fold scores are not independent observations and should not be used for naive confidence intervals; statistical comparison of models is the subject of the next session.

The DVC remote is a local directory, so a reader on another machine must rebuild the data from the pinned Hugging Face revision instead of running dvc pull; a shared remote (for example a cloud bucket or a shared drive) would remove this step. The manifest flag git_dirty = true should also be eliminated in the final thesis pipeline by recording the Git state before DVC starts writing outputs. Finally, the results were produced with Python 3.13.13 and scikit-learn 1.9.1; they also reproduced with Python 3.12.3 and scikit-learn 1.8.0, but other versions may change the last digits, which is why the Dockerfile pins the original environment.

References

Chalkidis, I., Jana, A., Hartung, D., Bommarito, M., Androutsopoulos, I., Katz, D. M., & Aletras, N. (2022). LexGLUE: A benchmark dataset for legal language understanding in English. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 4310–4330). Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.acl-long.297

Gaur, L. (2026). *The reproducibility crisis in AI & ML* [Course slides]. Research Methods and Scientific Integrity in AI and Advanced Technologies, Universidad Nacional Mayor de San Marcos.

Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns, 4*(9), Article 100804. https://doi.org/10.1016/j.patter.2023.100804

MLflow Project. (2026). *MLflow tracking and backend stores*. https://mlflow.org/docs/latest/tracking/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché-Buc, F., Fox, E., & Larochelle, H. (2021). Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program. *Journal of Machine Learning Research, 22*(164), 1–20.

Tuggener, D., von Däniken, P., Peetz, T., & Cieliebak, M. (2020). LEDGAR: A large-scale multi-label corpus for text classification of legal provisions in contracts. In *Proceedings of the 12th Language Resources and Evaluation Conference* (pp. 1235–1241). European Language Resources Association.

Appendix A. Git Commit Plan

Table A1
Commits of the Deliverable No. 5 Folder

Commit	Message
b766016	D5-00 Remove broken gitlink for 05_Reproducible_ML_Pipeline
1781bc1	D5-01 Initialize project structure, parameters and pinned dependencies
bca8150	D5-02 Add LEDGAR data preparation from pinned Hub revision and DVC pipeline
d0a6294	D5-03 Add seeded leakage-free clause classification pipeline
b39f3cd	D5-04 Run seven-seed study with baseline, MLflow logging and figures
dc37afd	D5-05 Add Dockerfile matching Python 3.13.13
e7973cb	D5-06 Add automated stranger test and reproduction README
2b7025a	D5-07 Regenerate results from clean committed state
ee0e171	D5-08 Configure DVC remote and push data

Note. Output of git log --oneline, oldest commit at the top. Repository: https://github.com/jesusincacardenas-crypto/UNMSM-Research-Methods-2026-II

Appendix B. Exact Reproduction Commands

```
# Option A: Docker (identical environment)
git clone https://github.com/jesusincacardenas-crypto/UNMSM-Research-Methods-2026-
II.git
cd UNMSM-Research-Methods-2026-II/05_Reproducible_ML_Pipeline
docker build -t d5-repro .
docker run --rm d5-repro   # prints REPRODUCED

# Option B: local environment with DVC
python -m venv .venv
.venv\Scripts\activate             # Linux/macOS: source .venv/bin/activate
pip install -r requirements.txt
dvc pull                   # same machine; elsewhere: dvc repro
dvc repro
python src/verify_reproduction.py --full
```

Expected reference (seed 42): accuracy mean 0.92367 and macro-F1 mean 0.92421. Expected dataset MD5: bc995ef9bacb814a07d06dbecad47ad0. Expected seven-seed result: macro-F1 0.92391 ± 0.00107.